



## MOBILNE APLIKACIJE

### Vežbe 7

### Mobilne komunikacije

2025/2026

# Sadržaj

1. JSON
2. REST web servis
  - 2.1 Poziv REST web servisa koristeći Retrofit biblioteku
    - Instanca retrofit-a
    - Definisanje REST servisa
    - Server
    - Postavljanje putanje do servera
    - Instanciranje REST servisa
    - Pozivanje REST servisa
4. Domaći

## 1. JSON

JSON je format za lakšu razmenu podataka. Podaci se zapisuju kao parovi ključ/vrednost. Ključ se navodi kao tekst pod duplim navodnicima nakon čega sledi vrednost.

```
"firstName":"John"
```

JSON vrednosti mogu biti:

- A number (integer or floating point)
- A string (in double quotes)
- A Boolean (true or false)
- An array (in square brackets)
- An object (in curly braces)
- Null

JSON objekat se zapisuje u parovima ključ/vrednost koji se nalaze unutar vitičastih zagrada:

```
{"firstName":"John", "lastName":"Doe"}
```

JSON Array sadrži ključ, nakon čega sledi niz elemenata u uglastim zgradama:

```
"employees":[
    {"firstName":"John", "lastName":"Doe"},
    {"firstName":"Anna", "lastName":"Smith"},
    {"firstName":"Peter", "lastName":"Jones"}
]
```

Na adresi <http://www.jsonschema2pojo.org> mogu se izgenerisati Java klase kopiranjem JSON sadržaja.

Potrebno je ispratiti par koraka:

- U *Package* delu napisati pun naziv paketa u projektu
- Za *Class name* dati naziv klasi koju generišemo
- Za *Source type* izabrati JSON
- Za *Annotation style* izabrati Gson
- [Opciono] *Preview* daje prikaz generisanih klasa
- Zip opcija preuzima zip fajl sa generisanim klasama i paketima koje možete importovati u projekat prostim *Copy-Paste* mehanizmom

- [Opciono] Ostala svojstva (*check boxovi*) mozete izmeniti po potrebi

## 2. REST web servis

Fokus REST servisa je na resursima i kako omogućiti pristup tim resursima. Resurs može biti predstavljen kao objekat, datoteka na disku, podaci iz aplikacije, baze podataka itd.

Prilikom dizajniranja sistema prvo je potrebno da identifikujemo resurse i da ustanovimo kako su međusobno povezani. Ovaj postupak je sličan modelovanju baze podataka.

Kada smo identifikovali resurse, sledeći korak je da pronađemo način kako da te resurse reprezentujemo u našem sistemu. Za te potrebe možemo koristiti bilo koji format za reprezentaciju resursa (npr. JSON).

Da bismo poslali zahtev nekom *web servisu* da dobijemo neki sadržaj, moramo napraviti jedan HTTP zahtev. HTTP zahtev se sastoji od nekoliko elemenata:

- <VERB> GET, PUT, POST, DELETE, OPTIONS itd. ili opis šta želimo da uradimo
- <URI> Putanja do resursa nad kojim će operacija biti izvedena

Nakon svakog HTTP zahteva sledi i HTTP odgovor od servera klijentu tj. onome ko je zahtev poslao. Klijent kao odgovor dobija sadržaj i statusni kod. Ovaj kod nam govori da li je naša operacija izvršena uspešno ili ne. Kodovi su reprezentovani celim brojevima i to:

- *Success 2xx* - sve je prošlo ok.
- *Redirection 3xx* - desila se redirekcija na neki drugi servis.
- *Error 4xx* - javila se greška na klijenskoj strani.
- *Error 5xx* - javila se greška na serverskoj strani.

Svaki servis mora imati neku adresu na koju šaljemo HTTP zahtev.

Ako želimo da dobavimo sve resurse vezane za osobe, adresa bi izgledala ovako:

Primer: <http://MyService/Persons>

Ako želim da dobavimo jednu osobu, u adresu ćemo dodati *path* parametar koji predstavlja id osobe:

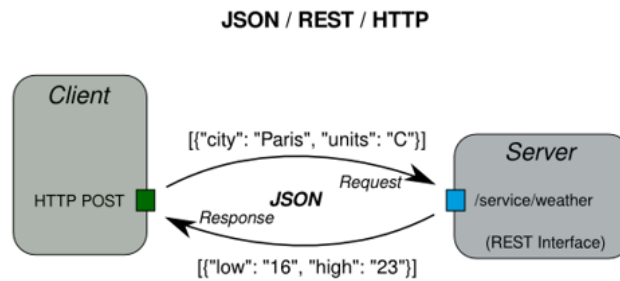
Primer: <http://MyService/Persons/1>

Ako želimo da ubacimo na primer: sortiranje ili filtriranje, prosledićemo *query* parametre, to možemo da uradimo tako što dodamo ? simbol na kraj putanje. Nakon specijalnog simbola, slede parovi ključ-vrednost spojeni & simbolom, ako tih parametara ima više od jednog.

Primer: <http://MyService/Persons?sort=ASC&filter=mobile>

Servisi koje pozivamo preko interneta imaju već definisanu strukturu (tačnu putanju, metod kojim se pozivaju, podatke koje očekuju, kako se pretražuju). Jedino što je potrebno da uradimo

je da nađemo konkretan servis i pogledamo kako on tačno očekuje da vršimo komunikaciju sa njim.



Slika 1. Primer komunikacije između klijenta i servera

## 2.1 Poziv REST web servisa koristeći *Retrofit* biblioteku

Da bismo instalirali *Retrofit* biblioteku potrebno je dodati nekoliko biblioteka u *build.gradle* datoteci.

```
49 implementation 'com.google.code.gson:gson:2.8.7'
50 implementation 'com.squareup.retrofit2:retrofit:2.3.0'
51 implementation 'com.squareup.retrofit2:converter-gson:2.3.0'
52 implementation 'com.squareup.okhttp3:logging-interceptor:3.12.1'
53
```

Slika 2. Dodavanje biblioteka

### Instanca retrofit-a

Kada su biblioteke instalirane, potrebno je definisati *retrofit* instancu koja će vršiti HTTP zahteve ka određenom REST servisu. U klasi *ClientUtils* kreirana je instanca retrofit-a (slika 3).

```
37 public static Retrofit retrofit = new Retrofit.Builder()
38     .baseUrl(SERVICE_API_PATH)
39     .addConverterFactory(GsonConverterFactory.create())
40     .client(test())
41     .build();
42
```

Slika 3. Kreiranje *retrofit* instance

## Definisanje REST servisa

Za svaki servis koji želimo da pozovemo moramo da definišemo metod sa kojim se poziva (GET, POST..), ali i dodatne parametre upita, dodatne putanje, a u slučaju POST zahteva i sadržaj koji je opisan JSON mapiranom Java klasom (u našem primeru klasa *Product*).

Na slici 4 nalazi se primer *ProductService* interfejsa koji sadrži metode *getAll*, *getById*, *add*, *deleteById*, *edit*. Ove metode opisuju koju metodu koristimo i šta očekujemo kao rezultat.

```
21 public interface ProductService {
22
23     @Headers({
24         "User-Agent: Mobile-Android",
25         "Content-Type:application/json"
26     })
27     @GET("product/")
28     Call<ArrayList<Product>> getAll();
29
30     @Headers({
31         "User-Agent: Mobile-Android",
32         "Content-Type:application/json"
33     })
34     @GET("product/{id}")
35     Call<Product> getById(@Path("id") Long id);
36
37     @Headers({
38         "User-Agent: Mobile-Android",
39         "Content-Type:application/json"
40     })
41     @POST("product/")
42     Call<Product> add(@Body Product product);
43 }
```

Slika 4. Interfejs *ProductService*

Spisak dostupnih anotacija, koje *retrofit* podržava, se nalazi na slici 5.

| Annotation | Description                                                                                                    |
|------------|----------------------------------------------------------------------------------------------------------------|
| @Path      | variable substitution for the API endpoint (i.e. username will be swapped for {username} in the URL endpoint). |
| @Query     | specifies the query key name with the value of the annotated parameter.                                        |
| @Body      | payload for the POST call (serialized from a Java object to a JSON string)                                     |
| @Header    | specifies the header with the value of the annotated parameter                                                 |

Slika 5. Anotacije i njihova značenja

Ukoliko neki servis, koji želimo da pozovemo, zahteva dodatne elemente, koje treba poslati unutar zaglavlja HTTP zahteva, to možemo definisati koristeći *Header* anotaciju. Svi elementi koji se navode su parovi ključ-vrednost (slika 6).

```
@Headers({
    "User-Agent: Mobile-Android",
    "Content-Type:application/json"
})
@DELETE("product/{id}")
```

Slika 6. Primer zaglavlja

## Server

Server koji pozivamo izgleda ovako:

`http://<service_ip_adress>:<service_port>/api/product`

Server je implementiran unutar projekta *shop-app-server* kao *Spring Boot* aplikacija. Projekat sadrži klasu *Product* sa poljima *id*, *title*, *description* i *imagePath*.

Na slici 7. vidi se primer REST kontrolera. Kontroler poseduje implementacije GET, POST, PUT, DELETE metoda.

```

16  no usages  ± Svetlana Antešević
17  @CrossOrigin
18  @RestController
19  @RequestMapping("/product")
20  public class ProductController {
21      5 usages
22      @Autowired
23      private ProductService productService;
24
25      no usages  ± Svetlana Antešević
26      @ApiOperation(value = "Get All Products Example", notes = "Returns all products from database.")
27      @ApiResponses(value = {
28          @ApiResponse(code = 200, message = "Successful returned products.", response = ArrayList.class)
29      })
30      @GetMapping
31      public ResponseEntity<?> getAll(){
32          System.out.println("GetAll controller method");
33          return new ResponseEntity<>(productService.getAll(), HttpStatus.OK);
34      }
35
36      no usages  ± Svetlana Antešević
37      @ApiOperation(value = "Get Product Example", notes = "Returns product with id from database.")
38      @ApiResponses(value = {
39          @ApiResponse(code = 200, message = "Successful returned product.", response = ProductDto.class),
40          @ApiResponse(code = 404, message = "Error", response = String.class)
41      })
42      @GetMapping("/{id}")
43      public ResponseEntity<?> getById(@PathVariable("id") Long id){
44          try{...}catch (Exception e){...}
45      }
46

```

Slika 7. REST kontroler implementiran u Java projektu

Na serverskoj strani kreirana je i API dokumentacija pomocu Swagger-a i dostupna je na linku <http://localhost:8080/api/swagger-ui/index.html#/>

## Postavljanje putanje do servera

Kako bi klijent (mobilna aplikacija) i server (Spring Boot aplikacija) mogli da ostvare komunikaciju, potrebno je povezati ih na istu mrežu. Pronaći IP adresu uređaja gde je server pokrenut. Preko IP adrese i porta možemo da pristupimo serveru.

Primer: <http://192.168.0.26:8080>

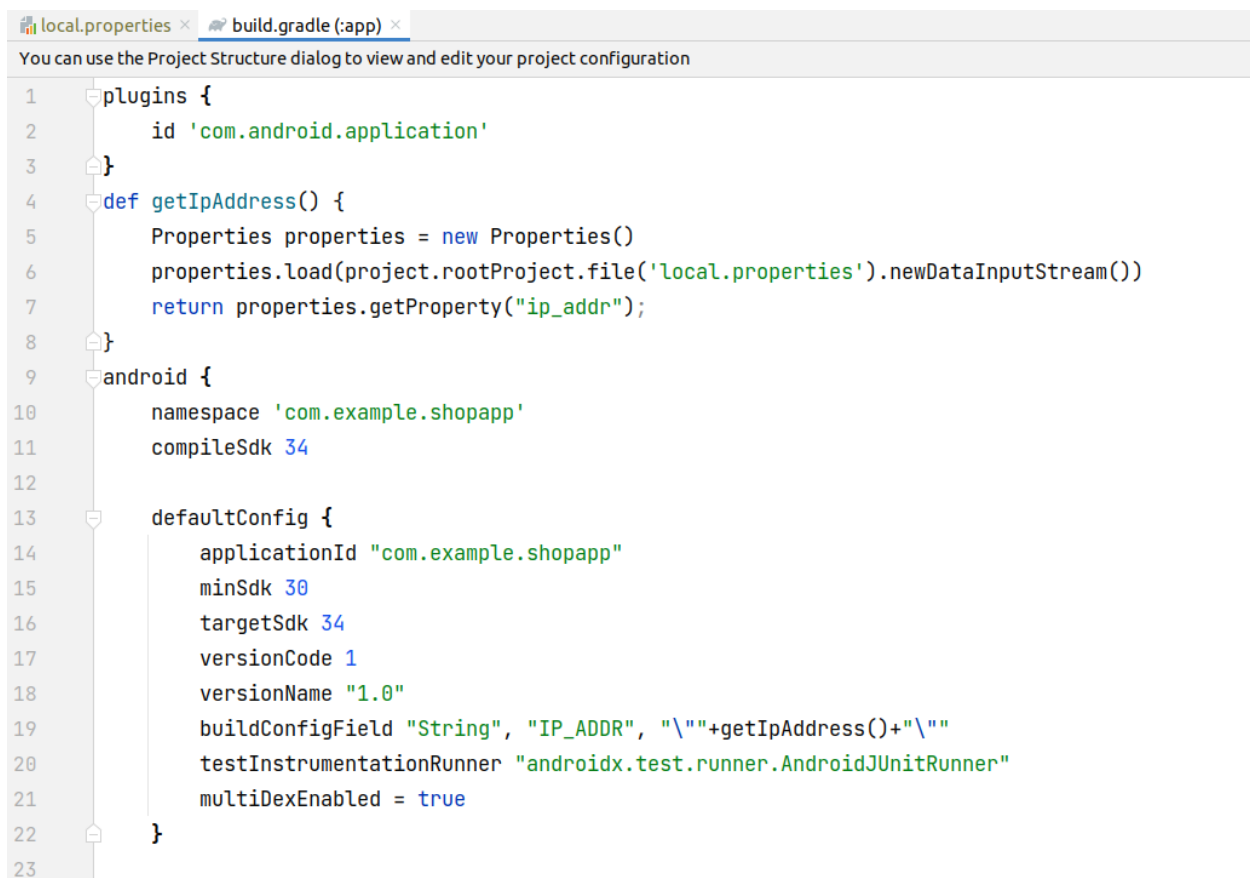
Kako IP adresu ne želimo da postavimo na git potrebno je da je sačuvamo u *local.properties* datoteku koju ne postavljamo na git repozitorijum (slika 8).



```
1  ## This file must *NOT* be checked into Version Control Systems,
2  # as it contains information specific to your local configuration.
3  #
4  # Location of the SDK. This is only used by Gradle.
5  # For customization when using a Version Control System, please read the
6  # header note.
7  #Sun Nov 12 11:39:17 CET 2023
8  sdk.dir=/home/svetlana/Android/Sdk
9  ip_addr=192.168.0.26
```

Slika 8: *local.properties* datoteka

Kako bismo dalje u kodu mogli da koristimo ovu *ip\_addr* varijablu moramo da izvršimo sledeću konfiguraciju u *build.gradle* datoteci. Linije koda od 5-9 i 20 (slika 9).



```
1  plugins {
2      id 'com.android.application'
3  }
4  def getIpAddress() {
5      Properties properties = new Properties()
6      properties.load(project.rootProject.file('local.properties').newDataInputStream())
7      return properties.getProperty("ip_addr");
8  }
9  android {
10     namespace 'com.example.shopapp'
11     compileSdk 34
12
13     defaultConfig {
14         applicationId "com.example.shopapp"
15         minSdk 30
16         targetSdk 34
17         versionCode 1
18         versionName "1.0"
19         buildConfigField "String", "IP_ADDR", "\"" + getIpAddress() + "\""
20         testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"
21         multiDexEnabled = true
22     }
23 }
```

Slika 9: *build.gradle* datoteka

Nakon toga možemo da pristupimo našem parametru preko *BuildConfig* klase na sledeći način *BuildConfig.IP\_ADDR* (slika 10). Na slici je takođe prikazana *ClientUtils* klasa u okviru koje je definisana putanja do servera.

```
11
12 public class ClientUtils {
13
14     //EXAMPLE: http://192.168.43.73:8080/api/
15     public static final String SERVICE_API_PATH = "http://" + BuildConfig.IP_ADDR + ":8080/api/";
16
17     /↴
```

Slika 10. Definisan server

## Instanciranje REST servisa

Nakon definisanja putanje servera, kreiranja *retrofit*-a i definisanja REST servisa, potrebno je izvršiti i instanciranje REST servisa u *ClientUtil* klasi (slika 11).

```
47      public static ProductService productService = retrofit.create(ProductService.class);
```

Slika 11. Instanciranje *ProductService*-a

## Pozivanje REST servisa

U *ProductListFragmentManagerModelu* nalazi se kod koji obrađuje odgovor REST servisa nakon GET poziva (slika 12). Ovim pozivom tražimo listu svih proizvoda.

Sam poziv REST servisa se odvija u pozadini i mi ne moramo da vodimo računa o tome, samo je potrebno da registrujemo šta da se desi kada odgovor stigne do nas. Taj deo se implementira dodavanjem *Callback<ArrayList<Product>>* unutar *enqueue* metode (slika 9). U *if*-u proveravamo da li je statusni kod HTTP odgovora jednak 200 i u tom slučaju dobavljamo listu proizvoda iz body dela i pozivamo adapter da smesti listu u *ListView*.

```
Call<ArrayList<Product>> call = ClientUtils.productService.getAll();
call.enqueue(new Callback<ArrayList<Product>>() {

    @Override
    public void onResponse(Call<ArrayList<Product>> call, Response<ArrayList<Product>> response) {
        if (response.code() == 200){
            Log.d( tag: "REZ", msg: "Meesage recieved");
            System.out.println(response.body());
            products = response.body();
            adapter = new ProductListAdapter(getActivity(), getActivity().getSupportFragmentManager(), products);
            setListAdapter(adapter);
            adapter.notifyDataSetChanged();
        }else{
            Log.d( tag: "REZ", msg: "Meesage recieved: "+response.code());
        }
    }

    @Override
    public void onFailure(Call<ArrayList<Product>> call, Throwable t) {
        Log.d( tag: "REZ", t.getMessage() != null?t.getMessage():"error");
    }
})
```

Slika 12. Logovanje poruka u zavisnosti od HTTP odgovora

## 4. Domaći

Domaći se nalazi na *Canvas-u* (*canvas.ftn.uns.ac.rs*) na putanji *Вежбе/07 Задатак.pdf*.

Primer možete preuzeti na sledećem linku:

<https://gitlab.com/mobilne-aplikacije/mobilne-aplikacije>

Za dodatna pitanja možete se obratiti asistentima:

- Milan Jovkić ([milan.jovkic@uns.ac.rs](mailto:milan.jovkic@uns.ac.rs))
- Jelena Matković ([matkovic.jelena@uns.ac.rs](mailto:matkovic.jelena@uns.ac.rs))